Use OpenSSL to generate keys and encrypt/decrypt files.

1. What is OpenSSL?

OpenSSL is a command-line tool used for:

- Encryption & decryption
- Generating keys (private & public)
- Creating digital certificates
- Hashing and signing data

Used by: Websites, VPNs, Linux servers, and cloud systems for secure data transfer.

2. Practical: Symmetric Encryption (AES)

In symmetric encryption, the same key is used for encryption and decryption.

Step 1 — Create a sample file

```
echo "This is a secret message" > secret.txt
```

Step 2 — Encrypt the file using AES-256-CBC

```
openssl enc -aes-256-cbc -salt -in secret.txt -out secret.enc
```

It will ask:

```
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
```

Enter a password (example: `1234`), then retype it.

A new file `secret.enc` will be created — this is the **encrypted** file (unreadable text).

Step 3 — Decrypt the file back to normal

```
openssl enc -aes-256-cbc -d -in secret.enc -out decrypted.txt
```

Enter the **same password** you used before.

Step-4- Now check the decrypted file:

cat decrypted.txt

3. Practical: Asymmetric Encryption (RSA)

In **asymmetric encryption**, you use:

- A **public key** (to encrypt data)
- A **private key** (to decrypt data)

Step 1 — Generate a Private Key

openssl genpkey -algorithm RSA -out private.pem -aes256

It will ask for a passphrase — enter any password (this protects your private key).

Step 2 — Extract the Public Key

openssl pkey -in private.pem -out public.pem -pubout

Step 3 — Create a message

echo "Asymmetric encryption example" > message.txt

Step 4 — Encrypt the file using **public key**

openssl pkeyutl -encrypt -pubin -inkey public.pem -in message.txt -out encrypted.bin

Step 5 — Decrypt the file using private key

openssl pkeyutl -decrypt -inkey private.pem -in encrypted.bin -out decrypted.txt

Now view the decrypted content:

cat decrypted.txt

Hash a password with SHA-256 and verify integrity.

1. What is Hashing?

Definition:
Hashing converts data into a fixed-length string (called a *hash value* or *digest*) using mathematical functions.

Example analogy:
It's like taking a fingerprint of your data — even if you change one letter, the fingerprint completely changes.

Common hashing algorithms: MD5, SHA-1, SHA-256, SHA-512

2. Practical Example: Hash a Password using SHA-256

Step 1 – Create a text file

```
echo "myStrongPassword123" > password.txt
```

Step 2 – Generate a SHA-256 hash

```
sha256sum password.txt
```

Output Example:

```
a83b9b5e0e414ab7df7d06cb15a9b8c31f54b9f9d40a5c711739dc98f42f3a5a
password.txt
```

The long string on the left is the hash value — a unique digital fingerprint of your password.

3. Verify File Integrity

Step 1 – Create a copy of the file

```
cp password.txt password_copy.txt
```

Step 2 – Hash both files

```
sha256sum password.txt password_copy.txt
```

Output Example:

```
a83b9b5e0e414ab7df7d06cb15a9b8c31f54b9f9d40a5c711739dc98f42f3a5a
password.txt

a83b9b5e0e414ab7df7d06cb15a9b8c31f54b9f9d40a5c711739dc98f42f3a5a
password_copy.txt
```

Step 3 – Change one file and check again

```
echo "extra text" >> password_copy.txt
```

sha256sum password_copy.txt

Output Example:

d1f2c9f3414d3b3c97d1258e5d08b1e8c8b5ef5b99b1ac53e03b80cb4b3e0ccf password_copy.txt